

EcoSort: A Multimodal Mobile Assistant for Household Waste Classification and Disposal Guidance

Dario Pomasqui
Yachay Tech
Urcuqui, Ecuador
dario.pomasqui@yachaytech.edu.ec

Abstract—EcoSort is a multimodal application that classifies household waste items by combining a photographic input with a short free-text hint provided by the user. The system returns a class label, calibrated confidence, and structured disposal guidance enriched by semantic rules. Implementation targets a mobile-first workflow where the client (Expo/React Native) uploads an image and a note to a FastAPI backend that blends a learned multimodal model with a semantic heuristic engine.

Index Terms—multimodal learning, waste classification, mobile AI, MobileNetV3, GRU

I. INTRODUCTION

Correct waste sorting is a small but impactful behavior that depends on contextual and often non-visual cues (contamination, reuse intent, safety). EcoSort aims to reduce uncertainty at the point of disposal by fusing visual information and short textual hints to produce a recommendation that is actionable for end users.

This report summarizes the dataset and preprocessing, the proposed multimodal architecture, training and validation strategy, deployment choices, and quantitative results obtained with two exported artifacts present in the repository (a Keras export and a PyTorch “pro” export).

II. METHODOLOGY

A. Input Modalities

The system ingests:

- **Image:** RGB photo resized and normalized for a MobileNetV3-Small backbone.
- **Text:** short user note tokenized with a compact vocabulary (SimpleTokenizer).

B. Dataset and Preprocessing

The consolidated dataset covers 12 target categories: battery, biological, brown-glass, cardboard, clothes, green-glass, metal, paper, plastic, shoes, trash and white-glass. The repository contains CSV annotations under `ml/artifacts/data/` (train/val/test splits). Images are resized, augmented during training (random crop, flip, color jitter) and normalized with ImageNet statistics. Text is normalized with a lightweight rule set and encoded into fixed-length token sequences (padding/truncation).

C. Model Architecture

The learned model implements late fusion of visual and textual features:

- **Image encoder:** MobileNetV3-Small (backbone), pooling to a 576-d feature vector. Optionally initialized with ImageNet weights and can be frozen during training.
- **Text encoder:** Embedding layer followed by a bidirectional GRU (hidden projection 96 per direction, output dim 192).
- **Fusion:** Concatenation of image (576) and text (192) features followed by a dense fusion block (256-d bottleneck, LayerNorm, GELU, dropout) and a final linear classifier producing logits for 12 classes.

Representative hyperparameters found in the repository artifacts: image size 128–160, text embedding dim 96, fusion dim 256, dropout 0.25–0.3, and vocab sizes in the hundreds (e.g., 213 tokens for the `keras_pro` export).

D. Training and Validation

Training scripts use PyTorch and Keras variants. Key training choices:

- **Optimizer:** AdamW.
- **Loss:** Cross-entropy with optional class weights (to mitigate imbalance).
- **LR schedule:** ReduceLROnPlateau based on validation F1/accuracy.
- **Regularization:** dropout, gradient clipping and early stopping with patience.

During Keras training the pipeline exports `best_model.keras`, `tokenizer.json` and `training_summary.json` with per-epoch histories. For PyTorch training the `best_model.pt` checkpoint and a structured `training_summary.json` are generated.

E. Backend and Mobile Integration

The mobile client (Expo) communicates with a FastAPI backend (`backend/app/`) through a `/predict` endpoint. The backend initializes a service that can operate in several modes: pure heuristic (semantic rules), `hybrid_keras`, or `hybrid_ai` (PyTorch). When a learned engine is available the service blends model and heuristic probabilities using

configurable weights and text-boosting heuristics to improve robustness in ambiguous scenarios.

III. IMPLEMENTATION DETAILS

A. Código y módulos principales

El repositorio organiza el código en módulos claros. A continuación se describen los componentes más relevantes y su propósito:

- **Modelo (PyTorch / Keras):** implementado en `ml/src/ecosort/model.py` (clases `EcoSortMultimodalNet`, `ImageEncoder`, `TextEncoder`). El modelo concatena características visuales (`MobileNetV3-Small`) y de texto (`Embedding + Bi-GRU`) y aplica una capa de fusión seguida de la clasificación.
- **Preprocesamiento y dataset:** `ml/src/ecosort/dataset.py` y `ml/src/ecosort/prepare_data.py` cargan anotaciones CSV, aplican transformaciones a las imágenes (`resize`, `crop`, `normalización`) y tokenizan texto con `ml/src/ecosort/text_features.py` (clase `SimpleTokenizer`).
- **Entrenamiento:** flujos en `ml/src/ecosort/train.py` (PyTorch) y `ml/src/ecosort/keras_train.py` (Keras). Ambos exportan artefactos (`best_model.pt / best_model.keras`, `tokenizer.json`, `training_summary.json`) y historiales CSV.
- **Inferencia:** `ml/src/ecosort/inference.py` (PyTorch) y `ml/src/ecosort/keras_inference.py` (Keras) cargan checkpoints y tokenizadores, preparan entradas (imagen + texto) y devuelven probabilidades y recomendaciones.
- **Reglas semánticas:** implementadas en `backend/app/semantic_rules.py`; analizan el texto de usuario, detectan intenciones y ajustan probabilidades mediante heurísticas (text-boost, penalizaciones, enriquecimiento de etiquetas).
- **Servicio y API:** `backend/app/service.py` crea la lógica `EcoSortService` que combina heurística y modelo aprendido; `backend/app/main.py` expone el endpoint `/predict` usado por la app móvil.
- **Cliente móvil:** `mobile/src/api/client.ts` gestiona las solicitudes al backend y normaliza respuestas para la UI Expo/React Native.

B. Flujo de ejecución resumido

Al recibir una petición desde la app móvil: (1) la imagen y la nota se suben al endpoint `/predict`; (2) el backend genera primero una predicción heurística con `semantic_rules.py`; (3) si hay un modelo cargado, se obtiene la predicción aprendida; (4) se mezclan probabilidades con reglas de ponderado y text-boosting; (5) se responde con `label_key`, `confidence`, `probabilities` y un objeto `recommendation` con pasos y justificación.

IV. RESULTS

Results were collected from the artifacts included in the repository. Two representative artifacts are reported below:

TABLE I
TEST SET SUMMARY FOR EXPORTED ARTIFACTS FOUND IN
ML/ARTIFACTS

Artifact	Accuracy	Notes
Keras export (keras_pro)	0.9207	training_summary.json (epochs comp)
PyTorch pro (report)	0.9991	ml/artifacts/reports/pro/test.

The Keras export reached a test accuracy of approximately 0.9207 with test loss 0.2485 (see `ml/artifacts/models/keras_pro/training_summary.json`). A separate pro export placed under `ml/artifacts/reports/pro` contains a near-perfect test report (accuracy 0.9991, macro F1 0.9992) and a saved confusion matrix image. These differences indicate multiple experimental runs and export formats; the pro export likely corresponds to a stronger or overfit configuration (verify with the training logs in `ml/artifacts/models/pro/`).

Figure 1 shows the test confusion matrix produced for the pro export.

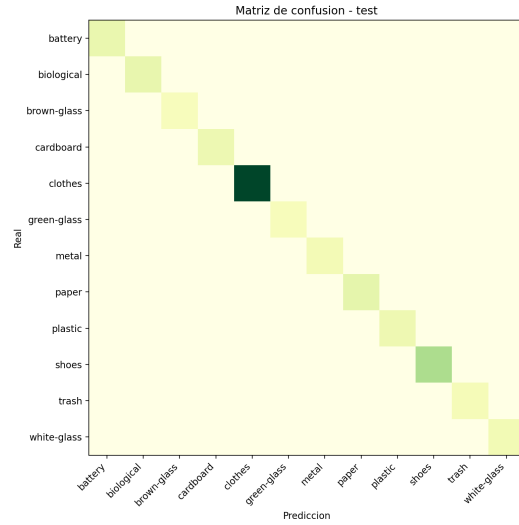


Fig. 1. Test confusion matrix for the pro export (file included in repository).

V. CONCLUSIONS AND FUTURE WORK

EcoSort demonstrates a practical multimodal pipeline that is deployable through a mobile client and a compact backend. The fusion of image and text improves context-aware disposal recommendations and enables a useful hybrid strategy where semantic heuristics complement learned predictions.

Limitations and next steps:

- Collect in-the-wild user notes to replace synthetic annotations and validate generalization.
- Audit and compare the different exported artifacts to ensure realistic generalization (the pro report suggests

potential overfitting). Use cross-validation and hold-out cities for robust evaluation.

- Export and benchmark a lightweight TFLite / ONNX runtime for on-device inference. The repository already contains an ONNX exporter (`ml/src/ecosort/export_onnx.py`) to assist this path.
- Improve the UI/UX to request clarifying text prompts when confidence is low and add a lightweight on-device fallback.

REFERENCES

- [1] A. Howard *et al.*, “Searching for MobileNetV3,” in *Proc. IEEE/CVF Int. Conf. Computer Vision*, 2019.
- [2] K. Cho *et al.*, “Learning Phrase Representations using RNN Encoder-Decoder,” 2014.
- [3] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems*, 2019.